MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 4

of Cryptography and Security

Topic: Block Ciphers. The DES Algorithm

Done by: st. gr. FAF-243, Augustin Carazanu
Checked by: *assist. univ. Maia Zaica*
FCIM, UTM

Chişinău – 2026

Purpose of the work

The purpose of this laboratory work is to study the DES (Data Encryption Standard) block cipher and to implement one of its elements. The task is chosen with the formula nr_task = n mod 11; my number in the group list is n = 3, so 3 mod 11 = 3 and I solved **Task 2.3: in the DES algorithm K+ is given; determine the round key Ki for a given i**. The program was written in Python 3 with a Tkinter interface. As required, it displays the tables used and all the intermediate steps, and the input can be entered by the user or generated randomly.

1. Short theory

1.1. The DES algorithm

DES is a symmetric block cipher adopted as a U.S. federal standard in 1977 (FIPS 46). It encrypts **64-bit blocks** with a **64-bit key**, of which only **56 bits** are used; every 8th bit is a parity bit. Encryption has three parts:

1. the **initial permutation IP** of the 64-bit block, which is then split into two halves L0 and R0 of 32 bits;

2. **16 rounds** of a Feistel network: Li = Ri−1 and Ri = Li−1 ⊕ f(Ri−1, Ki), where Ki is the 48-bit round key;

3. the halves are swapped (R16L16) and the **final permutation IP^{-1}** is applied.

The round function f(R, K) expands R from 32 to 48 bits with the expansion E, computes E(R) ⊕ K, splits the result into eight 6-bit groups B1…B8, replaces each group with 4 bits using the **S-boxes** S1…S8 (the only non-linear part of DES), and permutes the resulting 32 bits with P. Decryption uses the same algorithm with the round keys in reverse order, K16 … K1.

1.2. The key schedule

Every round needs its own 48-bit key Ki. All 16 round keys are derived from the main key K:

Step 1. PC-1. The 64-bit key K is permuted with Permuted Choice 1. PC-1 drops the 8 parity bits (8, 16, …, 64) and gives the 56-bit key **K+** (Table 1.1).

Table 1.1 – PC-1: position in K of each bit of K+

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22

14	6	61	53	45	37	29
21	13	5	28	20	12	4

Step 2. Split. K+ is split into two halves of 28 bits: **C0** (bits 1–28) and **D0** (bits 29–56).

Step 3. Left shifts. For n = 1 … 16, Cn and Dn are obtained from Cn−1 and Dn−1 by a circular left shift of 1 or 2 positions, according to Table 1.2. The total number of shifts is 28, so C16 = C0 and D16 = D0.

Table 1.2 – Number of left shifts in each round

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Step 4. PC-2. The two halves are joined into CnDn (56 bits) and Permuted Choice 2 selects 48 of them, in the order of Table 1.3. The result is the round key **Kn**. For example, the first bit of Kn is bit 14 of CnDn, the second is bit 17, and so on; bits 9, 18, 22, 25, 35, 38, 43 and 54 are never used.

Table 1.3 – PC-2: position in CnDn of each bit of Kn

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

In Task 2.3, K+ is already given, so Step 1 is not required. To obtain Ki, the program splits K+ (Step 2), applies the shifts of rounds 1 … i (Step 3) and then PC-2 (Step 4). As an extra option, it can also compute K+ from an 8-character key with PC-1.

2. Implementation

The program has two files: `des_keys.py` contains the DES tables and the key-schedule functions, and `gui.py` contains the Tkinter window. K+ can be typed as 56 binary digits or 14 hexadecimal digits, generated randomly, or computed from an 8-character key. The round i is chosen from 1 to 16. The main fragments are presented below.

2.1. The tables

```
PC2 = [14, 17, 11, 24, 1, 5,
       3, 28, 15, 6, 21, 10,
       23, 19, 12, 4, 26, 8,
       16, 7, 27, 20, 13, 2,
       41, 52, 31, 37, 47, 55,
       30, 40, 51, 45, 33, 48,
       44, 49, 39, 56, 34, 53,
       46, 42, 50, 36, 29, 32]

# Number of left shifts for rounds 1..16
SHIFTS = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]
```

PC-2 and the shift schedule are copied from the standard FIPS 46-3. Positions are numbered from 1, as in the standard.

2.2. Input validation

```
def parse_kplus(text):
    """K+ given as 56 binary digits or 14 hex digits (spaces allowed)."""
    s = "".join(text.split())
    if not s:
        raise ValueError("Enter K+ (56 bits in binary or 14 hexadecimal digits).")
    if set(s) <= {"0", "1"} and len(s) != 14:
        if len(s) != 56:
            raise ValueError(f"K+ in binary must have exactly 56 bits (it has
{len(s)}).")
        return s
    if all(c in "0123456789abcdefABCDEF" for c in s):
        if len(s) != 14:
            raise ValueError(f"K+ in hexadecimal must have exactly 14 digits (it has
{len(s)}).")
        return bin(int(s, 16))[2:].zfill(56)
    bad = next(c for c in s if c not in "0123456789abcdefABCDEF")
    raise ValueError(f"Invalid character '{bad}'. K+ may contain only 0/1 (56 bits) "
                     f"or hexadecimal digits 0-9, A-F (14 digits).")


def parse_round(value):
    v = str(value).strip()
    if not v.isdigit() or not 1 <= int(v) <= 16:
        raise ValueError(f"Invalid round '{v}'. i must be an integer from 1 to 16.")
    return int(v)
```

`parse_kplus()` removes the spaces and accepts K+ either as exactly 56 bits or as exactly 14 hexadecimal digits; for anything else it explains the correct format. `parse_round()` accepts only an integer from 1 to 16.

2.3. Permutation and circular left shift

```
def permute(bits, table):
    return "".join(bits[p - 1] for p in table)


def left_shift(bits, n):
    return bits[n:] + bits[:n]
```

`permute()` builds a new bit string by taking, for each entry of the table, the bit at that position (minus 1, because Python counts from 0). `left_shift()` moves the first n bits to the end, which is a circular left shift.

2.4. Computing Ki

```
def round_key(kplus, i):
    """Return K_i and every intermediate step needed to obtain it."""
    c, d = kplus[:28], kplus[28:]
    steps = [(0, 0, c, d)]
    for n in range(1, i + 1):
        c, d = left_shift(c, SHIFTS[n - 1]), left_shift(d, SHIFTS[n - 1])
        steps.append((n, SHIFTS[n - 1], c, d))
    cd = c + d
    return {"C0": kplus[:28], "D0": kplus[28:], "steps": steps,
            "CD": cd, "K": permute(cd, PC2)}


# ---------------------------------------------------------------
```

`round_key()` splits K+ into C0 and D0, applies the shifts of rounds 1 … i and keeps every intermediate pair (Cn, Dn) so that the interface can display them. Finally it joins Ci and Di and applies PC-2.

2.5. Generating K+ from an 8-character key

```
def random_kplus():
    return "".join(secrets.choice("01") for _ in range(56))


def key_to_bits(key):
    """8 ASCII characters -> 64 bits."""
    if len(key) != 8:
        raise ValueError(f"The key must have exactly 8 characters (it has
{len(key)}).")
    if any(ord(c) > 127 for c in key):
        raise ValueError("The key must contain only ASCII characters.")
    return "".join(format(ord(c), "08b") for c in key)


# ---------------------------------------------------------------
# Key schedule
# ---------------------------------------------------------------
def permute(bits, table):
    return "".join(bits[p - 1] for p in table)


def left_shift(bits, n):
    return bits[n:] + bits[:n]


def kplus_from_key(key):
    k64 = key_to_bits(key)
    return k64, permute(k64, PC1)
```

`random_kplus()` generates 56 random bits. `kplus_from_key()` converts 8 ASCII characters into 64 bits (8 bits per character) and applies PC-1.

3. Results

3.1. Check against the standard example

To check the program I used the well-known example key K = 133457799BBCDFF1 (hexadecimal), whose round keys are published in the DES literature (J. Orlin Grabbe, “The DES Algorithm Illustrated”). Its K+ is:

```
K+ = 1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111
   = F0CCAAF556678F (hex)
```

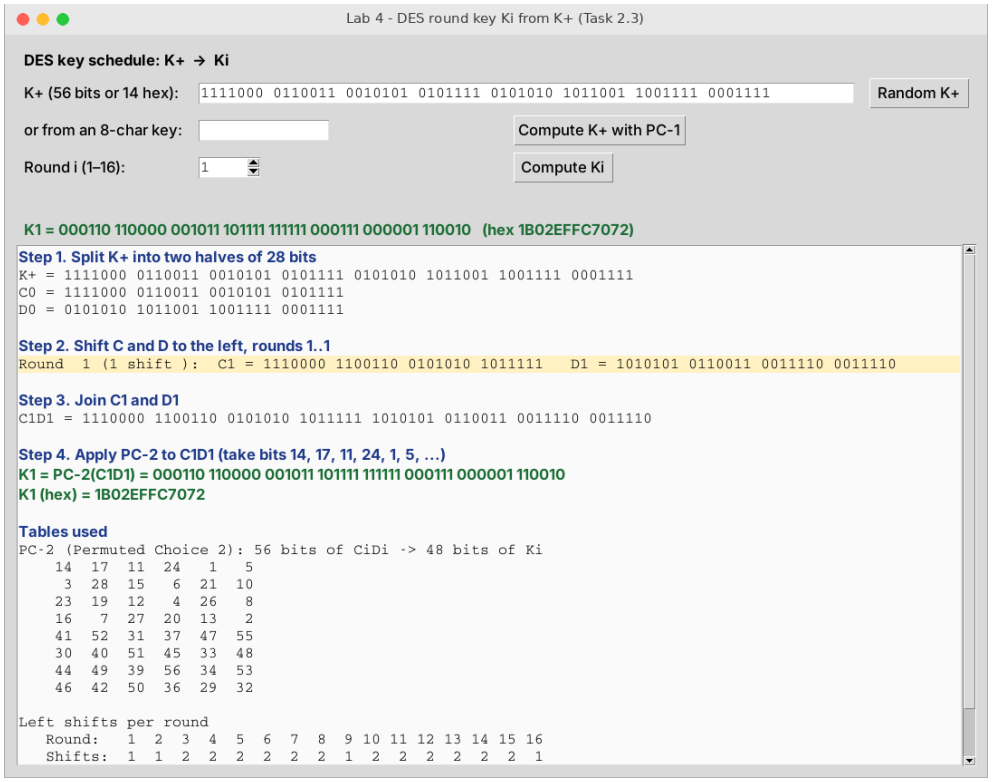


Figure 3.1 – K1 for the example K+

The program gives K1 = 000110 110000 001011 101111 111111 000111 000001 110010, the same value as in the literature. Manual check of the first bits with PC-2 and C1D1 (C1 = 1110000 1100110 0101010 1011111):

• bit 1 of K1 = bit 14 of C1D1 = 0; bit 2 = bit 17 = 0; bit 3 = bit 11 = 0;

• bit 4 = bit 24 = 1; bit 5 = bit 1 = 1; bit 6 = bit 5 = 0, so the first group is 000110, as computed by the program.

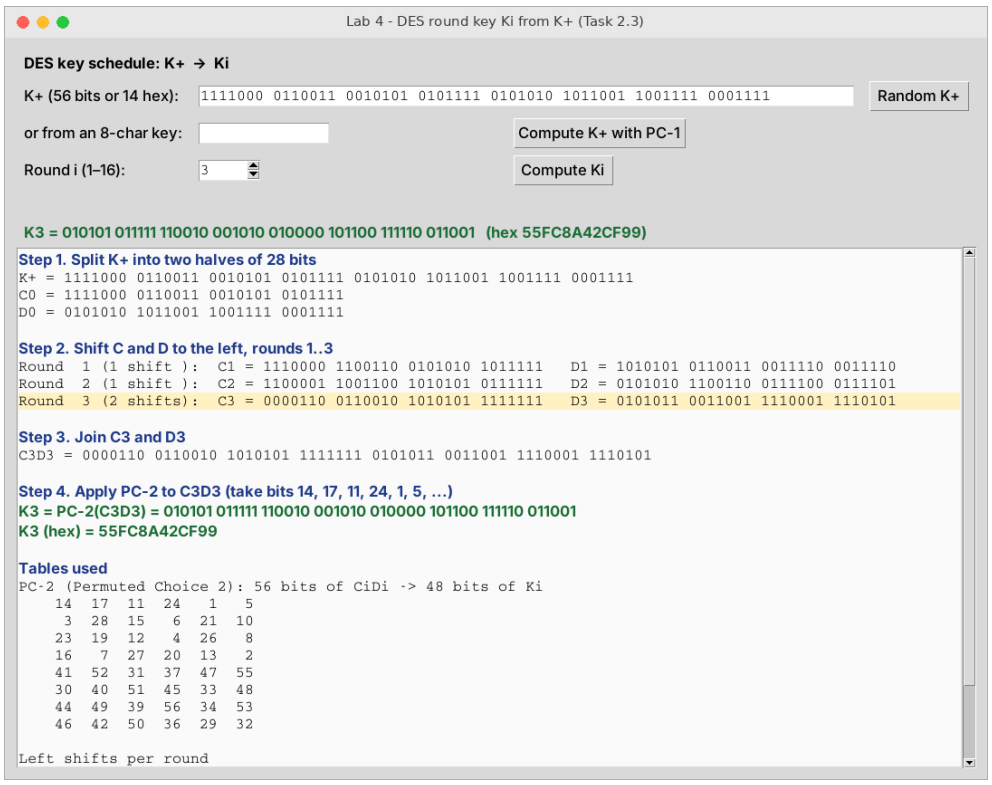


Figure 3.2 – K3: round 3 uses 2 shifts (highlighted row)

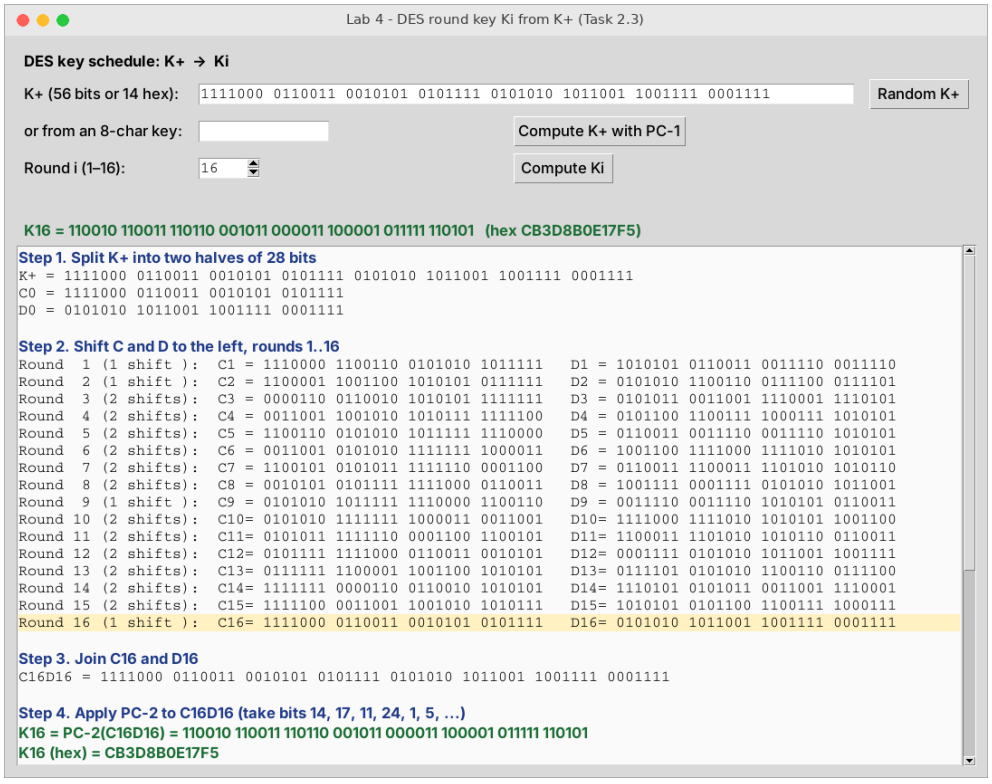


Figure 3.3 – K16: after 28 shifts in total, C16 = C0 and D16 = D0

K3 = 010101 011111 110010 001010 010000 101100 111110 011001 and K16 = 110010 110011 110110 001011 000011 100001 011111 110101 also match the published values. Figure 3.3 also shows that C16 = C0 and D16 = D0, because the shifts add up to 28.

3.2. K+ from an 8-character key and from random bits

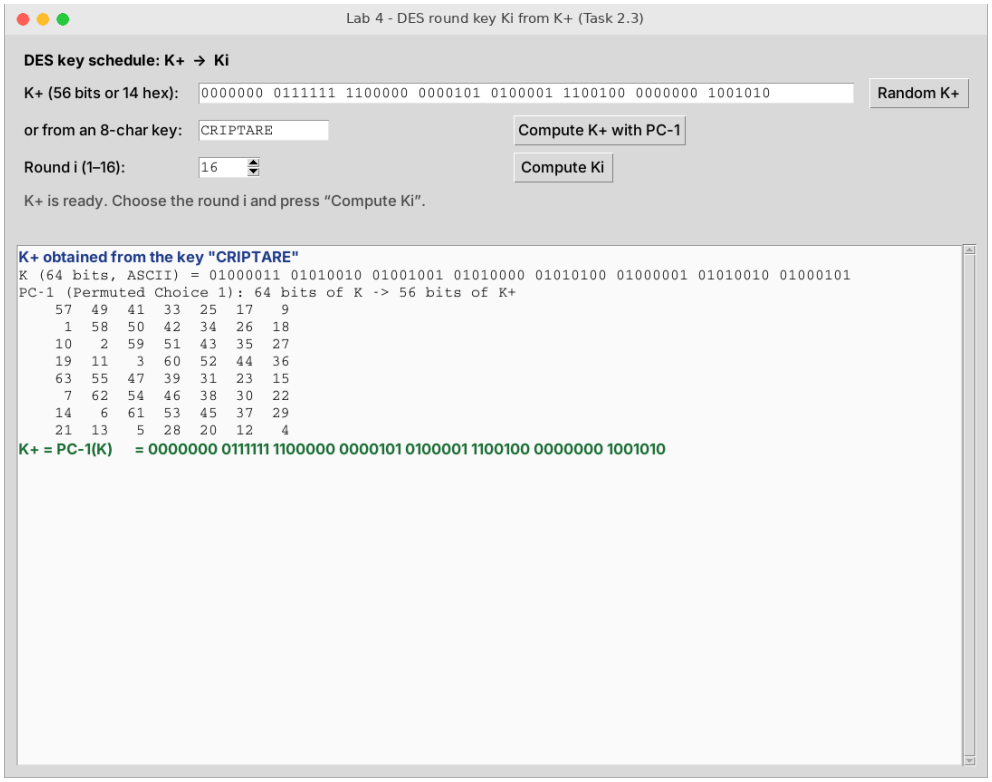


Figure 3.4 – K+ computed with PC-1 from the key "CRIPTARE"

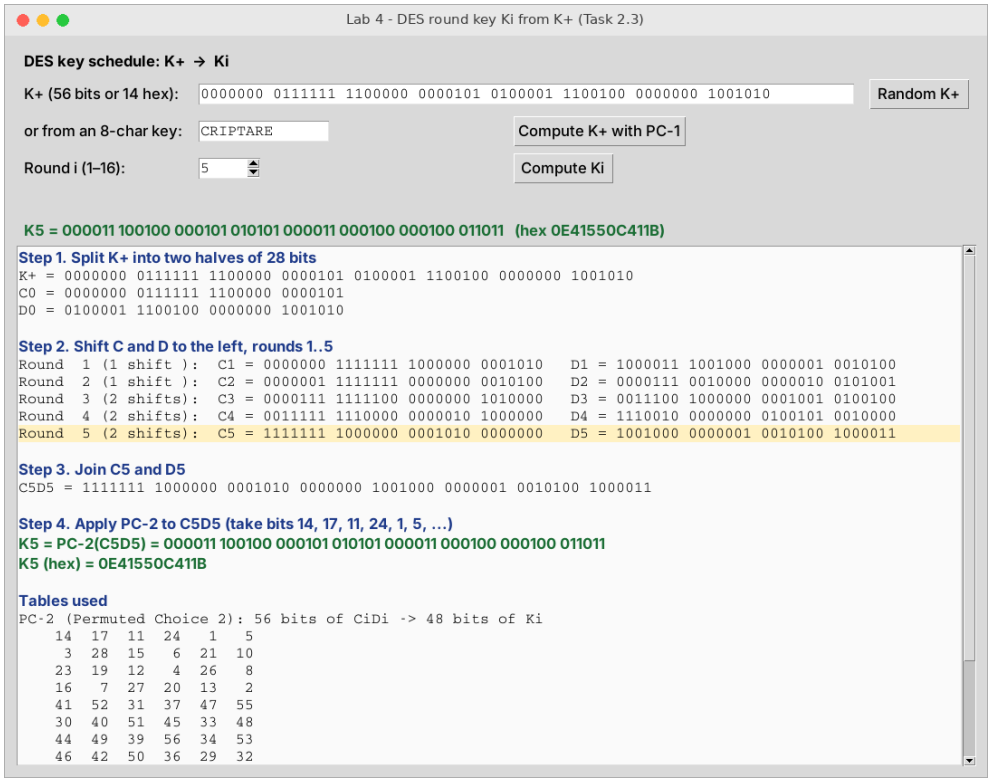


Figure 3.5 – K5 for the K+ of the key "CRIPTARE"

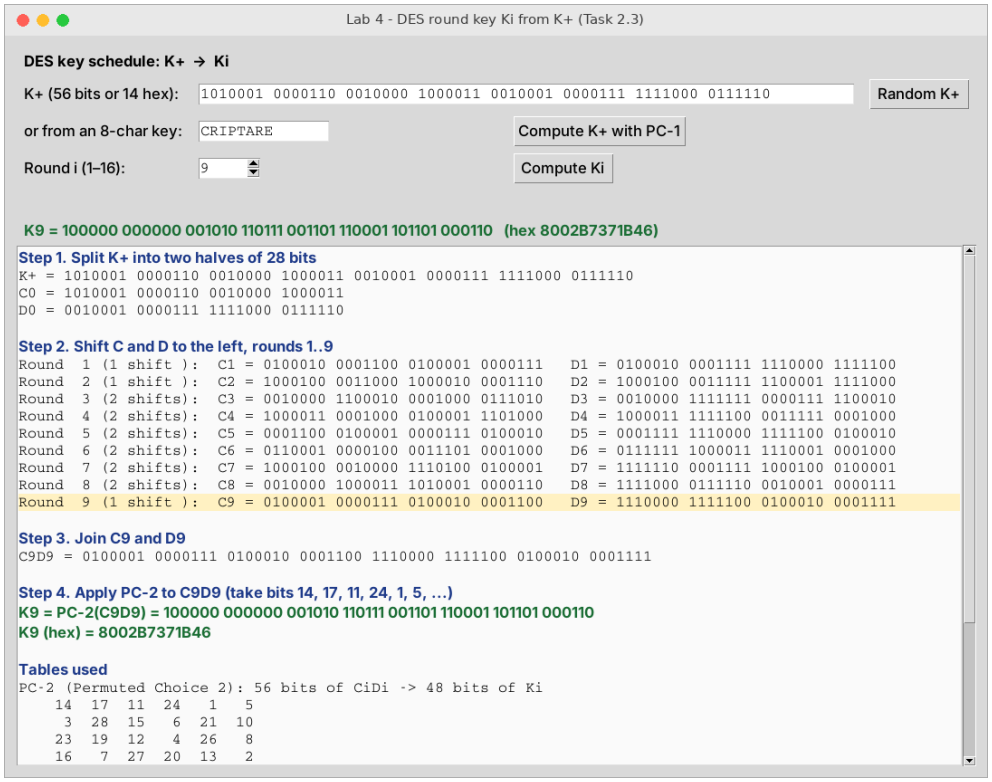


Figure 3.6 – K9 for a randomly generated K+

3.3. Invalid input

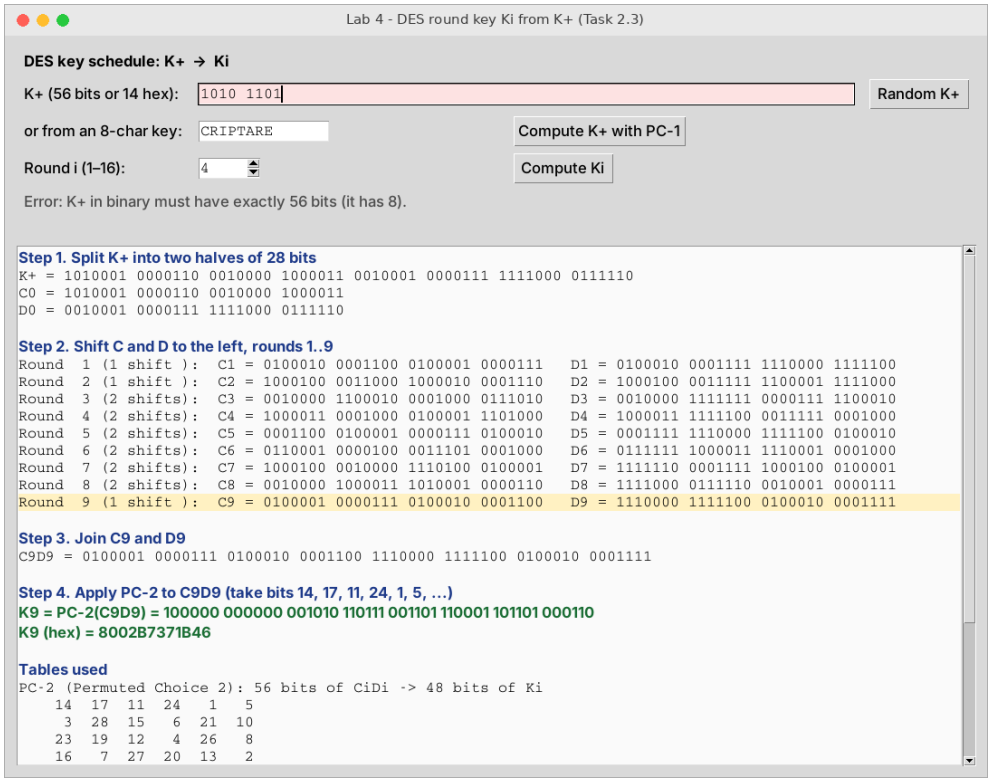


Figure 3.7 – K+ with a wrong number of bits

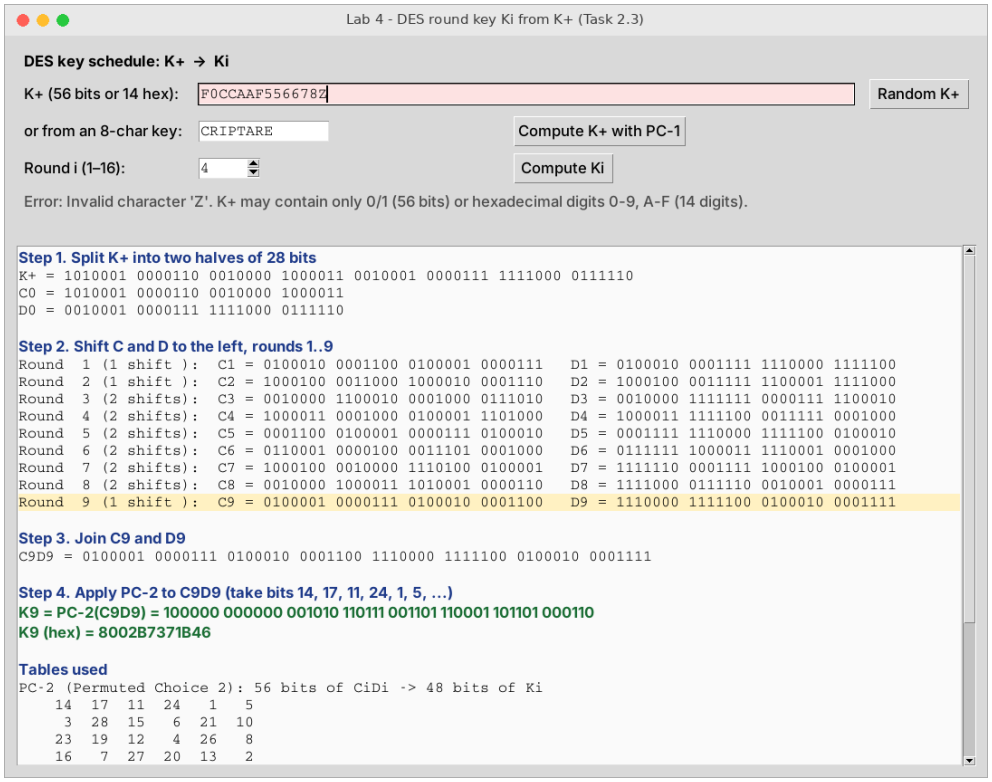


Figure 3.8 – K+ with an invalid character

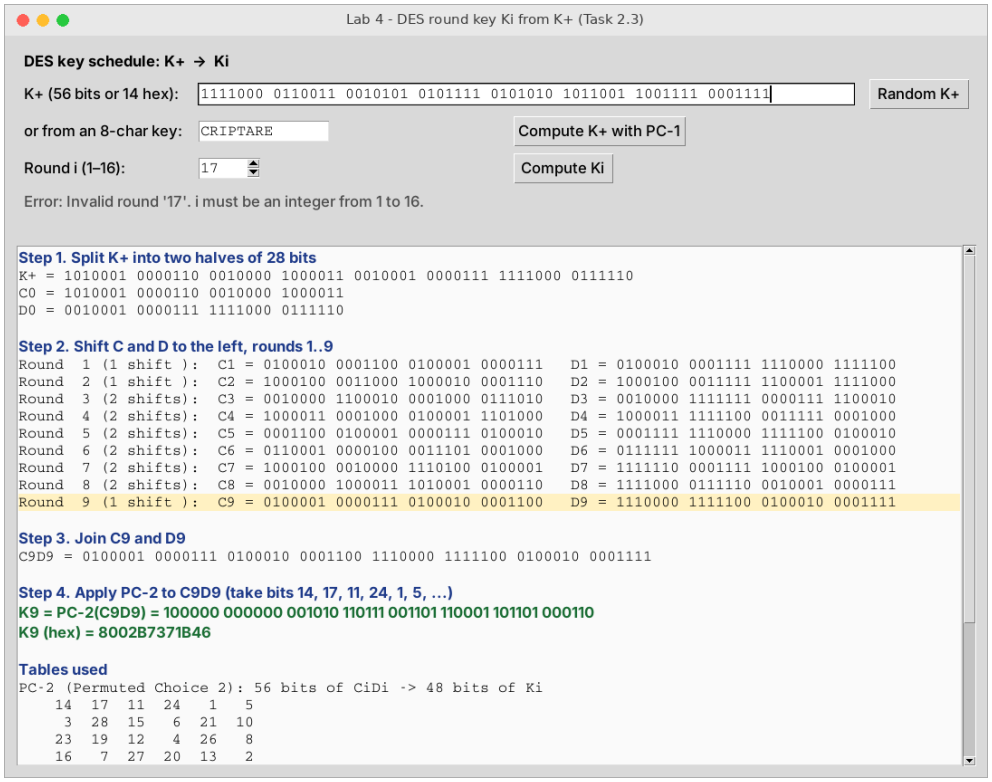


Figure 3.9 – A round number outside 1…16

4. Conclusions

The DES key schedule is simple and fast. From the 56-bit key K+, the two 28-bit halves C and D are rotated to the left by 1 or 2 positions in each round, and PC-2 selects 48 of the 56 bits to form the round key. Because the shifts add up to 28, after round 16 the halves come back to C0 and D0, which is what allows the same key schedule to be run in reverse for decryption.

Every round key uses a different selection of the key bits, so each of the 56 key bits takes part in about 14 of the 16 rounds. This spreads the influence of the key over the whole encryption. However, the effective key length of 56 bits is too small today: the $2^{56} \approx 7.2 \times 10^{16}$ possible keys were searched exhaustively already in 1998, which is why DES was replaced by Triple DES and then by AES.

During this work I implemented the permutations PC-1 and PC-2, the circular left shifts and the full derivation of a round key, and I checked the results against the values published for the standard example key.

5. Source code

The full source code is available on GitHub: https://github.com/Afyze/cs-2026-AugustinCarazanu/tree/main/lab4